

Gerador/Verificador de Assinaturas

Rafael Carvalho J. Araújo
*Departamento de
Ciência da Computação
Universidade de Brasília
Brasília, Brasil
222021817
rrafaelcarvalhoj@gmail.com*

João Victor P. Vieira
*Departamento de
Ciência da Computação
Universidade de Brasília
Brasília, Brasil
211036114
institutodnb@gmail.com*

Abstract—Este trabalho apresenta a implementação de um sistema completo de geração e verificação de assinaturas digitais baseado no algoritmo RSA. Foram implementadas do zero as primitivas de geração de chaves com teste de primalidade de Miller-Rabin, cifração/decifração RSA, padding OAEP com Rede de Feistel, hashing SHA-3 e formatação BASE64 no padrão PEM.

1. Introdução

A segurança da informação em ambientes digitais depende fundamentalmente de mecanismos que garantam autenticidade, integridade e não-repúdio nas comunicações. A assinatura digital é a primitiva criptográfica que responde a essas três exigências simultaneamente: ela vincula matematicamente um documento ao seu signatário de forma que qualquer adulteração posterior seja detectável e que a autoria não possa ser negada. Introduzido o RSA [1], proposto em 1978 por Rivest, Shamir e Adleman, permanece como um dos criptosistemas assimétricos mais amplamente empregados, presente em protocolos como TLS, SSH e S/MIME. Sua segurança repousa sobre a dificuldade computacional de fatorar o produto de dois primos grandes — problema para o qual não se conhece solução eficiente em computação clássica. A correta implementação de suas primitivas, contudo, exige atenção a detalhes que vão além da matemática central: a geração de primos criptograficamente seguros, a introdução de aleatoriedade controlada via padding e a serialização portátil do resultado.

Este trabalho descreve a implementação de um sistema completo de geração e verificação de assinaturas digitais RSA em C++11, sem o uso de bibliotecas de alto nível para as primitivas criptográficas. Foram implementadas do zero: a geração de primos com o teste probabilístico de Miller-Rabin [2], a cifração e decifração RSA via exponenciação modular, o esquema de preenchimento OAEP [3] com sua rede de Feistel interna, e a serialização em Base64 no formato PEM. O resumo criptográfico emprega o SHA-3 (Keccak) [4], padronizado pelo NIST em 2015, e a aritmética sobre inteiros de 2048 bits é delegada à biblioteca GMP.

O relatório está organizado da seguinte forma: a Seção 2 apresenta a fundamentação teórica das primitivas implementadas; a Seção 3 descreve o protocolo de assinatura e verificação como integração dessas primitivas; a Seção 4 detalha o ambiente experimental e analisa os resultados obtidos; e a Seção 5 apresenta as conclusões do trabalho.

1.1. Padrões Adotados

A arquitetura de software implementada neste trabalho não propõe primitivas criptográficas ad-hoc, mas sim a integração estrita de protocolos estabelecidos pela indústria e padronizados por órgãos internacionais. A cifração assimétrica e o esquema de preenchimento probabilístico (OAEP com MGF1) seguem a RFC 8017 (PKCS #1 v2.2). A geração de primos para as chaves adere aos critérios do FIPS 186-4, enquanto o resumo criptográfico implementa a construção esponja do Keccak conforme o FIPS 202 (NIST).

2. Fundamentação Teórica

2.1. Teste de Primalidade de Miller-Rabin

No contexto de assinatura de documentos, o algoritmo RSA é amplamente utilizado. Para que o RSA ofereça segurança adequada, a chave deve ser composta por dois primos p e q de pelo menos 1024 bits cada. A verificação de primalidade por divisão tentativa é computacionalmente inviável nessa escala, um número de 1024 bits possui aproximadamente 10^{308} dígitos decimais, tornando a busca exaustiva por fatores impraticável mesmo com hardware moderno. O teste de Miller-Rabin [2] resolve o problema de verificação de primalidade com custo computacional polinomial no número de bits.

O teste parte do Pequeno Teorema de Fermat: para qualquer primo p e inteiro a não divisível por p ,

$$a^{p-1} \equiv 1 \pmod{p}. \quad (1)$$

Caso a congruência (1) falhe para alguma base a , então p certamente não é primo. O problema é que a recíproca não

é verdadeira: existem compostos que satisfazem a equação para certas bases, os chamados *números de Carmichael*, enganando o teste de Fermat simples. O teste de Miller-Rabin supera essa limitação por meio de uma decomposição mais refinada.

2.1.1. Decomposição $n - 1 = 2^r \cdot d$. Dado um candidato ímpar n , escreve-se $n - 1$ fatorando todos os fatores 2:

$$n - 1 = 2^r \cdot d, \quad d \text{ ímpar.} \quad (2)$$

Por exemplo, para $n = 13$: $n - 1 = 12 = 2^2 \cdot 3$, logo $r = 2$ e $d = 3$. Essa decomposição transforma a verificação em uma sequência de quadrados modulares capaz de detectar compostos que passariam pelo critério de Fermat.

2.1.2. Critério de Aprovação. Para uma base a escolhida aleatoriamente em $[2, n - 2]$, calcula-se $x = a^d \bmod n$. O candidato n passa na rodada se ao menos uma das seguintes condições for satisfeita:

$$x \equiv 1 \pmod{n}, \quad (3)$$

$$x^{2^j} \equiv -1 \pmod{n} \quad \text{para algum } j \in \{0, 1, \dots, r - 1\}. \quad (4)$$

Caso nenhuma das condições (3) ou (4) seja satisfeita, n é **definitivamente composto**. O teste é repetido k vezes com bases independentes.

2.1.3. Probabilidade de Erro. Cada rodada com uma base aleatória tem probabilidade no máximo $1/4$ de classificar erroneamente um composto como primo [2]. Com k rodadas independentes, a probabilidade de erro é limitada por

$$P(\text{erro}) \leq \left(\frac{1}{4}\right)^k. \quad (5)$$

A Tabela 1 apresenta o decaimento da probabilidade de erro em função do número de rodadas.

TABLE 1. PROBABILIDADE DE ERRO DO TESTE DE MILLER-RABIN EM FUNÇÃO DO NÚMERO DE RODADAS.

Rodadas k	Probabilidade de erro
10	$\approx 10^{-6}$
20	$\approx 10^{-12}$
40	$\approx 10^{-24}$

Com $k = 40$, a probabilidade teórica de erro é limitada por 4^{-40} , valor negligenciável para fins práticos neste projeto.

2.1.4. Geração do Candidato a Primo. A geração de primos de 1024 bits segue o procedimento abaixo:

- 1) Gera-se um número aleatório de exatamente 1024 bits.
- 2) O bit mais significativo é forçado a 1, garantindo que o número possua exatamente 1024 bits e não degenere para 1023 bits.

- 3) O bit menos significativo é forçado a 1, garantindo que o candidato seja ímpar.
- 4) Aplica-se o teste de Miller-Rabin com 40 rodadas.
- 5) Caso o candidato seja rejeitado, soma-se 2 e retorna-se ao passo 4.

O passo 2 é essencial: sem ele, um número de 1024 bits poderia ter o bit mais significativo igual a zero e ser, na prática, um número de 1023 bits, gerando um módulo $n = p \cdot q$ menor que o esperado e comprometendo o nível de segurança do sistema.

2.2. Criptografia Assimétrica RSA

O RSA [1] é um criptossistema de chave pública cuja segurança repousa sobre a dificuldade computacional de fatorar o produto de dois primos grandes. O sistema opera com um par de chaves matematicamente ligadas: o que uma cifra, somente a outra decifra.

2.2.1. Geração de Chaves. O procedimento de geração de chaves compreende as seguintes etapas. Primeiro, geram-se dois primos p e q de pelo menos 1024 bits cada, utilizando o teste de Miller-Rabin descrito na Seção anterior. Em seguida, calcula-se o módulo público

$$n = p \cdot q, \quad (6)$$

cujo comprimento é de aproximadamente 2048 bits. A função totiente de Euler é então obtida por

$$\varphi(n) = (p - 1)(q - 1). \quad (7)$$

O expoente público adotado é $e = 65537$, valor padrão na indústria por três razões: trata-se do quarto primo de Fermat ($2^{16} + 1$), possui apenas dois bits iguais a 1 em representação binária, o que acelera a exponenciação modular, e é suficientemente grande para resistir a ataques de baixo expoente. Por fim, o expoente privado é calculado como o inverso modular de e ,

$$d \equiv e^{-1} \pmod{\varphi(n)}, \quad (8)$$

obtido por meio do algoritmo de Euclides Estendido. A chave pública é o par (e, n) e a chave privada é o par (d, n) .

2.2.2. Operações Fundamentais. Cifrar e decifrar são a mesma operação matemática; o que difere é a chave empregada. Dado um bloco de mensagem M com $0 \leq M < n$, o texto cifrado C é obtido por

$$C = M^e \bmod n, \quad (9)$$

e a mensagem original é recuperada por

$$M = C^d \bmod n. \quad (10)$$

A corretude das operações (9) e (10) é garantida pelo Teorema de Euler: como $e \cdot d \equiv 1 \pmod{\varphi(n)}$, tem-se

$$M^{e \cdot d} \equiv M \pmod{n}. \quad (11)$$

2.2.3. RSA na Assinatura Digital. A assinatura digital com RSA resulta da inversão dos papéis das chaves. O signatário, único detentor da chave privada, produz a assinatura S sobre o hash H do documento por

$$S = H^d \bmod n, \quad (12)$$

enquanto qualquer verificador, de posse da chave pública, pode confirmar a autenticidade computando

$$H = S^e \bmod n \quad (13)$$

e comparando o resultado com o hash recalculado do documento recebido. Essa assimetria garante que apenas o titular da chave privada possa produzir uma assinatura válida, ao passo que a verificação permanece pública e irrestrita.

O RSA puro, contudo, é determinístico: a mesma mensagem sempre produz a mesma assinatura, tornando o esquema suscetível a ataques de texto escolhido e de falsificação. O esquema de padding OAEP, apresentado na Seção 2.4, introduz aleatoriedade controlada e resolve essa limitação.

2.3. Função de Hash SHA-3 (Keccak)

Uma função de hash criptográfica mapeia uma entrada de tamanho arbitrário para uma saída de tamanho fixo. Para ser considerada segura, deve satisfazer quatro propriedades fundamentais [5]: resistência à pré-imagem (dado h , é inviável encontrar m tal que $\text{hash}(m) = h$); resistência à segunda pré-imagem (dado m , é inviável encontrar $m' \neq m$ com o mesmo hash); resistência à colisão (é inviável encontrar qualquer par $m \neq m'$ com o mesmo hash); e efeito avalanche (uma alteração de 1 bit na entrada modifica aproximadamente 50% dos bits da saída).

2.3.1. Motivação para o SHA-3. O SHA-1 foi formalmente comprometido em 2017 pelo ataque SHAttered. O SHA-2, embora ainda seguro, compartilha a mesma estrutura interna de seus predecessores (construção de Merkle-Damgård), o que torna ambos suscetíveis a classes semelhantes de ataque. Em 2007, o NIST iniciou uma competição pública em busca de um padrão estruturalmente distinto. Em 2012, o algoritmo Keccak foi selecionado [4] e padronizado como SHA-3 por meio do FIPS 202 [5], publicado em agosto de 2015.

2.3.2. Construção Esponja. O diferencial do Keccak em relação a todos os padrões anteriores é a *construção esponja* [4], composta por duas fases sequenciais.

Na fase de **absorção**, os blocos da mensagem são incorporados ao estado interno: cada bloco é combinado via XOR com a porção de *rate* do estado, seguido da aplicação de uma função de permutação f . Na fase de **espremida**, o hash é extraído do estado, também r bits por vez, com aplicações intercaladas de f , até que o comprimento de saída desejado seja atingido.

O estado interno é uma matriz 5×5 de palavras de 64 bits, totalizando 1600 bits, dividida em duas regiões:

- **Rate r :** porção que recebe os dados de entrada — 1088 bits na variante SHA3-256;
- **Capacity c :** porção protegida que fundamenta a segurança — 512 bits na variante SHA3-256, com $r + c = 1600$.

A cada bloco absorvido, a permutação f executa 24 rodadas compostas por cinco operações ($\theta, \rho, \pi, \chi, \iota$) que difundem os bits por todo o estado, garantindo o efeito avalanche.

2.3.3. SHA-3 no Contexto do Projeto. O SHA3-256 produz sempre uma saída de 32 bytes (256 bits), independentemente do tamanho do documento, dimensão compatível com o bloco RSA de 128 bytes empregado com OAEP. A função é utilizada em dois momentos distintos no sistema:

- 1) **Assinatura:** o documento passa pelo SHA3-256, gerando um hash de 32 bytes que é então submetido ao esquema OAEP e cifrado com RSA.
- 2) **Internamente ao OAEP:** a função MGF1 utiliza o SHA3-256 para derivar as máscaras empregadas no padding, conforme descrito na Seção 2.4.

A implementação delega o cálculo ao Crypto++, única biblioteca externa autorizada para esta primitiva.

2.4. OAEP - Optimal Asymmetric Encryption Padding

O RSA puro apresenta duas vulnerabilidades estruturais decorrentes de seu determinismo. Em primeiro lugar, a mesma mensagem sempre produz o mesmo texto cifrado, o que permite a um atacante construir um dicionário de pares mensagem-cifrado. Em segundo lugar, o esquema é maleável: dado $C = M^e \bmod n$, é possível forjar $C' = C \cdot 2^e \bmod n$, que decifra para $2M$ sem que o atacante conheça M . O OAEP, proposto por Bellare e Rogaway [3] e padronizado no PKCS#1 v2.2 [6], resolve ambas as vulnerabilidades introduzindo aleatoriedade controlada e estrutura verificável antes da cifração RSA.

2.4.1. Estrutura do Bloco. Antes de ser submetida ao RSA, a mensagem é transformada em um bloco de tamanho fixo de 128 bytes (para RSA-2048):

$$EM = 0 \times 00 \parallel \text{maskedSeed} \parallel \text{maskedDB}, \quad (14)$$

onde *maskedSeed* ocupa 32 bytes e *maskedDB* ocupa 94 bytes. O campo interno *DB* é definido como

$$DB = lHash \parallel PS \parallel 0 \times 01 \parallel H, \quad (15)$$

com $lHash = \text{SHA3-256}("")$ (hash da string vazia, atuando como marcador de integridade), PS composto por zeros de preenchimento até completar o bloco, e H o hash SHA3-256 do documento. A composição completa do bloco é apresentada na Tabela 2.

TABLE 2. ESTRUTURA DO BLOCO *EM* NO ESQUEMA OAEP PARA RSA-2048 (128 BYTES).

0x00	<i>maskedSeed</i>	<i>lHash</i>	<i>PS</i> 0x01	<i>H</i>
1 byte	32 bytes	32 bytes	variável	32 bytes
<i>maskedDB</i> — 94 bytes				

2.4.2. MGF1 — Mask Generation Function. O MGF1 é o componente central do OAEP: dada uma semente de tamanho fixo, gera uma máscara de comprimento arbitrário concatenando saídas sucessivas de SHA3-256. Cada iteração aplica a função sobre a semente concatenada com um contador inteiro crescente, truncando o resultado ao comprimento necessário e gerando um resultado pseudorandom. Formalmente, $\text{MGF1}(\text{seed}, \ell)$ produz os primeiros ℓ bytes de $\text{SHA3-256}(\text{seed}||0)||\text{SHA3-256}(\text{seed}||1)||\dots$, conforme especificado por Moriarty [6].

2.4.3. Rede de Feistel — Codificação. A codificação OAEP aplica uma rede de Feistel onde cada metade mascara a outra. O processo opera da seguinte forma:

- 1) Gera-se um valor aleatório *seed* de 32 bytes a partir de `/dev/urandom`.
- 2) Calcula-se $\text{dbMask} = \text{MGF1}(\text{seed}, 94)$ e obtém-se $\text{maskedDB} = \text{DB} \oplus \text{dbMask}$.
- 3) Calcula-se $\text{seedMask} = \text{MGF1}(\text{maskedDB}, 32)$ e obtém-se $\text{maskedSeed} = \text{seed} \oplus \text{seedMask}$.
- 4) Concatenam-se os campos conforme a equação (14) para formar o bloco *EM*, que é então cifrado com RSA.

A aleatoriedade do *seed* garante que o mesmo hash produza um bloco *EM* diferente a cada execução, eliminando o determinismo do RSA puro.

2.4.4. Rede de Feistel — Decodificação. A decodificação desfaz as operações em ordem inversa:

- 1) Decifra-se *C* com a chave privada para recuperar *EM* e separam-se os campos *maskedSeed* e *maskedDB*.
- 2) Recupera-se $\text{seed} = \text{maskedSeed} \oplus \text{MGF1}(\text{maskedDB}, 32)$.
- 3) Recupera-se $\text{DB} = \text{maskedDB} \oplus \text{MGF1}(\text{seed}, 94)$.
- 4) Verifica-se a integridade do bloco e extrai-se o hash *H*.

2.4.5. Verificação de Integridade. Dois mecanismos independentes detectam adulteração antes da comparação de hashes. Primeiro, os 32 bytes iniciais de *DB* devem ser exatamente $\text{SHA3-256}(\text{" "})$: qualquer alteração no bloco RSA corromperia esse campo. Segundo, o delimitador 0x01 deve estar presente após os zeros de preenchimento; sua ausência indica bloco corrompido. Se qualquer verificação falhar, o sistema lança exceção antes de prosseguir à comparação final de hashes.

3. Assinatura Digital

A assinatura digital combina as primitivas descritas nas seções anteriores em um protocolo coeso: o SHA3-256 produz um resumo do documento, o OAEP protege esse resumo contra manipulação, e o RSA aplica a chave privada para produzir um valor verificável por qualquer detentor da chave pública correspondente. O resultado é um mecanismo que vincula matematicamente um documento a seu signatário, sem exigir que as partes compartilhem segredos. As subseções seguintes detalham as garantias oferecidas e o pipeline de execução implementado.

3.1. O que uma Assinatura Digital Garante

Uma assinatura digital produzida pelo sistema oferece três garantias fundamentais. A **autenticidade** decorre do uso exclusivo da chave privada *d* na geração da assinatura. Apenas o titular do par de chaves pode produzi-la. A **integridade** é assegurada pelo hash SHA3-256 do documento: qualquer alteração, por mínima que seja, modifica o hash e invalida a assinatura na verificação. O **não-repúdio** resulta da assimetria do par: a chave pública *e* verifica mas não cria, o signatário não pode negar a autoria de uma assinatura válida.

3.2. Fluxo de Assinatura

O pipeline completo de assinatura encadeia as primitivas descritas nas seções anteriores em quatro etapas sequenciais, ilustradas na Figura 1.

A primeira etapa computa o hash SHA3-256 do documento, produzindo um resumo de 32 bytes independente do tamanho do arquivo (`sha3.cpp`). Em seguida, o hash é submetido ao OAEP: uma semente aleatória de 32 bytes é obtida de `/dev/urandom` e a rede de Feistel produz um bloco blindado de 128 bytes (`padding_oaep.cpp`). O bloco é então elevado à chave privada via $\text{bloco}^d \bmod n$, operação implementada em `rsa_core.cpp` sob o nome `decrypt_raw()`. Por fim, o resultado — um número de aproximadamente 2048 bits — é serializado em bytes e codificado em Base64, gerando o arquivo `contrato.sig` (`formatter.cpp`).

Vale observar que a função `decrypt_raw()` é utilizada para *assinar*, não para decifrar. O nome reflete a simetria matemática do RSA: assinar e decifrar são a mesma operação — aplicar o expoente privado *d* — e o que diferencia os dois usos é apenas o contexto semântico. A inversão dos papéis das chaves, descrita na Seção 2.2, é precisamente o mecanismo que torna a assinatura possível.

3.3. Fluxo de Verificação

O processo de verificação ilustrado na Figura 2 reconstrói o hash original a partir da assinatura e o compara com o hash do documento recebido. A validade é estabelecida em duas camadas independentes: a primeira ocorre dentro

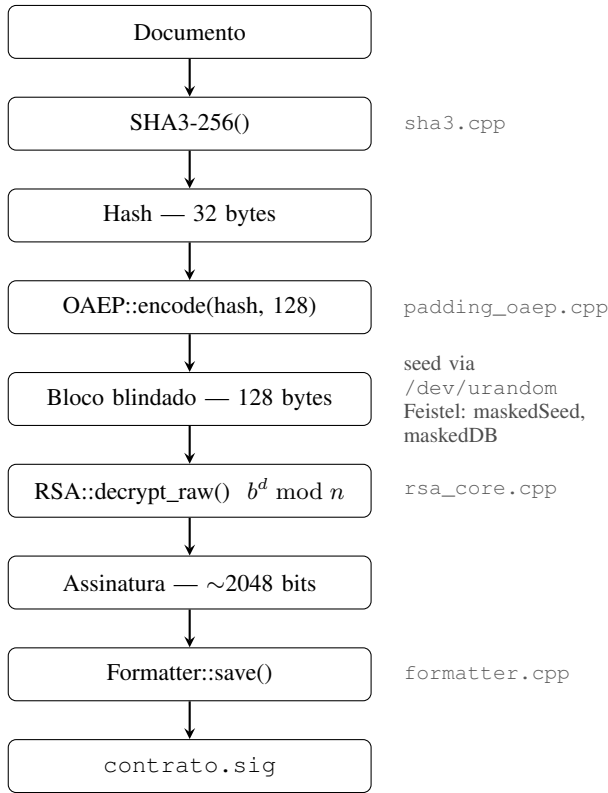


Figure 1. Pipeline completo de assinatura digital.

do OAEP, que lança exceção imediatamente se o bloco estiver corrompido, antes mesmo de qualquer comparação de hashes; a segunda é a comparação final entre o hash recuperado e o hash recalculado sobre o documento recebido.

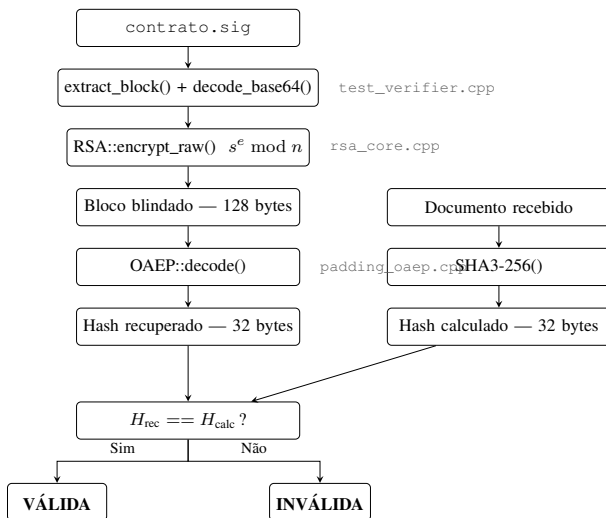


Figure 2. Pipeline de verificação de assinatura digital.

3.4. Formato do Arquivo .sig

A saída da operação modular na assinatura RSA é um bloco maciço de dados binários puros (*raw binary*) correspondente a 2048 bits. Se armazenado diretamente em disco, esse bloco invariavelmente conterá caracteres de controle não imprimíveis e sequências que não pertencem à tabela ASCII padrão ou ao UTF-8. A transmissão desses dados brutos por protocolos de rede baseados em texto (como SMTP para e-mails ou HTTP) frequentemente resulta em uma corrupção silente de bytes, pois os sistemas operacionais tentam interpretar ou corrigir esses caracteres invisíveis.

Devido ao rigor do determinismo criptográfico abordado nas seções anteriores, a alteração ou perda de um único bit durante o transporte corromperia a fechadura matemática, resultando em uma falha de verificação na máquina do destinatário.

Para mitigar esse problema e garantir a portabilidade da assinatura, o sistema serializa o resultado numérico em um vetor de bytes e aplica a codificação Base64, padronizada pela RFC 4648. A codificação atua como uma camada de envelopamento segura — convertendo blocos de 3 bytes binários em 4 caracteres alfanuméricos estritamente imprimíveis e seguros.

O arquivo de extensão `.sig` gerado pelo sistema armazena exclusivamente essa representação em Base64, frequentemente estruturada no padrão visual PEM (*Privacy-Enhanced Mail*). Dessa forma, a integridade da assinatura matemática é preservada na camada de aplicação, permitindo que o arquivo seja copiado, anexado e interpretado por qualquer sistema operacional ou editor de texto sem o risco de mutação de *encoding*.

4. Ambiente Experimental e Análise de Resultados

4.1. Descrição do Cenário

Os experimentos foram conduzidos no ambiente descrito na Tabela 3. O sistema operacional Linux é requisito funcional: a geração do *seed* aleatório do OAEP depende de `/dev/urandom`, disponível apenas em ambientes Unix.

TABLE 3. ESPECIFICAÇÕES DO AMBIENTE EXPERIMENTAL.

Componente	Especificação
CPU	Intel Core i7-13700H (20 threads) @ 4.8 GHz
Memória RAM	16.0 GB
Sistema Operacional	Ubuntu 24.04.4 LTS (kernel 6.17.0)
Linguagem	C++11
Biblioteca GMP	libgmp-dev — aritmética modular
Biblioteca Crypto++	libcrypto++-dev — SHA-3 e Base64
Compilação	g++ via Makefile, flag <code>-O2</code>

4.2. Análise de Resultados

4.2.1. Geração de Chaves — Teste de Miller-Rabin. A geração de chaves é a etapa de maior custo computacional e

de maior variância, pois depende do número de candidatos rejeitados antes de encontrar dois primos de 1024 bits. A Tabela 4 apresenta os tempos medidos em 10 execuções completas do assinador.

TABLE 4. TEMPOS DE GERAÇÃO DE CHAVES EM 10 EXECUÇÕES.

Execução	Tempo (s)
1	0,202
2	0,192
3	0,704
4	0,675
5	0,832
6	0,763
7	0,467
8	1,636
9	0,644
10	0,599
Média	0,671
Mínimo	0,192
Máximo	1,636

A variância observada reflete diretamente o comportamento estocástico do teste de Miller-Rabin: em casos favoráveis, o primeiro candidato gerado já é primo; em casos desfavoráveis, múltiplos candidatos são descartados antes de encontrar dois primos válidos de 1024 bits.

4.2.2. Assinatura com Documentos de Diferentes Tamanhos. O tempo de hashing com SHA-3 cresce linearmente com o tamanho do documento. Contudo, a operação RSA opera sempre sobre 128 bytes (o bloco OAEP contendo o hash), independentemente do tamanho do documento original. Isso confirma uma propriedade fundamental das assinaturas digitais: o custo criptográfico é constante. A Tabela 5 ilustra esse comportamento.

TABLE 5. TEMPO DE ASSINATURA POR TAMANHO DE DOCUMENTO.

Tamanho do documento	Tempo total (s)
1 byte	0,244
61 bytes	~0,671 (média)
500 bytes	0,771
100 KB	0,386
10 MB	1,719

A variação dominante continua sendo a geração de chaves, não o tamanho do documento. O tempo de SHA-3 sobre 10 MB é de aproximadamente 40 ms, o que é baixo em comparação ao custo médio de geração dos primos.

4.2.3. Verificação. A verificação é ordens de magnitude mais rápida que a assinatura, pois não envolve geração de primos. A Tabela 6 detalha o tempo de cada etapa para um documento de 61 bytes.

4.2.4. Casos de Falha e Comportamento nos Limites.
Documento adulterado: Alterando um único caractere do documento, o hash SHA-3 produz um valor completamente diferente. O verificador detecta a adulteração na comparação final e rejeita a assinatura. O efeito avalanche do SHA-3 garante que qualquer alteração, por menor que seja, invalida o resultado.

TABLE 6. TEMPO POR ETAPA NO PROCESSO DE VERIFICAÇÃO.

Etapa	Tempo
Parsing do .sig + Base64	< 1 ms
RSA com chave pública ($S^e \bmod n$)	~5 ms
Reversão OAEP	< 1 ms
SHA-3 do documento (61 bytes)	< 1 ms
Total (documento 61 bytes)	~8 ms
Total (documento 10 MB)	~68 ms

Arquivo .sig adulterado: Alterando um caractere na seção Base64 da assinatura, o RSA produz um número diferente ao reverter o bloco. O OAEP detecta a corrupção antes mesmo da comparação de hashes, ao verificar o primeiro byte do bloco decifrado. Esse comportamento demonstra a defesa em duas camadas: o OAEP atua como primeira barreira; a comparação de hashes é a segunda.

Limite de capacidade do OAEP: O OAEP com $block_size = 128$ bytes suporta no máximo 62 bytes de mensagem direta, conforme

$$cap = block_size - 2 \times h_{len} - 2 = 128 - 2 \times 32 - 2 = 62 \text{ bytes.} \quad (16)$$

A Tabela 7 apresenta o comportamento do sistema nos limites dessa capacidade.

TABLE 7. COMPORTAMENTO DO OAEP NOS LIMITES DE CAPACIDADE.

Tamanho da mensagem	Resultado
32 bytes (hash SHA-3)	Sucesso — 30 bytes abaixo do limite
62 bytes (limite exato)	Sucesso
63 bytes (1 acima do limite)	Exceção: mensagem excede capacidade do bloco

No sistema implementado, o OAEP recebe sempre 32 bytes (o hash SHA-3 do documento), deixando uma margem de 30 bytes em relação ao limite. O documento em si nunca é submetido ao OAEP; apenas sua impressão digital.

5. Conclusão

Este trabalho demonstrou que é possível implementar um sistema funcional e seguro de assinaturas digitais RSA a partir das primitivas matemáticas fundamentais, sem recorrer a bibliotecas criptográficas de alto nível. A integração entre o teste de Miller-Rabin, a cifragem RSA, o padding OAEP e o hash SHA-3 produziu um pipeline coeso capaz de assinar documentos de qualquer tamanho e verificar sua autenticidade com detecção robusta de adulteração.

Os experimentos evidenciaram que o custo computacional dominante é a geração de primos de 1024 bits, com tempo médio de 0,671 s e variância estocástica inerente ao caráter probabilístico do Miller-Rabin. A operação RSA em si — tanto na assinatura quanto na verificação — opera invariavelmente sobre um bloco de 128 bytes, tornando o custo criptográfico independente do tamanho do documento. A verificação, por não envolver geração de primos, ordens de magnitude mais rápida, completando-se em cerca de 8 ms para documentos pequenos.

Os testes de casos confirmaram a eficácia do design em camadas: adulterações no documento são detectadas pela comparação de hashes, enquanto adulterações na assinatura são interceptadas pelo OAEP antes mesmo dessa comparação, por meio da verificação do IHash e do delimitador 0x01. Essa defesa em profundidade reforça a robustez do sistema contra manipulações tanto no documento original quanto no arquivo de assinatura.

References

- [1] R. L. Rivest, A. Shamir, and L. Adleman, “A method for obtaining digital signatures and public-key cryptosystems,” *Commun. ACM*, vol. 21, no. 2, p. 120–126, Feb. 1978. [Online]. Available: <https://doi.org/10.1145/359340.359342>
- [2] M. O. Rabin, “Probabilistic algorithm for testing primality,” *Journal of Number Theory*, vol. 12, no. 1, pp. 128–138, 1980. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0022314X80900840>
- [3] M. Bellare and P. Rogaway, “Optimal asymmetric encryption,” in *Advances in Cryptology — EUROCRYPT 1994*, ser. Lecture Notes in Computer Science, A. De Santis, Ed., vol. 950. Berlin, Heidelberg: Springer, 1995, pp. 92–111.
- [4] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, “The Keccak Reference,” Submission to NIST SHA-3 Competition (Round 3), 2011. [Online]. Available: <https://keccak.team/files/Keccak-reference-3.0.pdf>
- [5] National Institute of Standards and Technology, “FIPS PUB 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions,” National Institute of Standards and Technology, Gaithersburg, MD, Federal Information Processing Standard 202, Aug. 2015. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.202.pdf>
- [6] K. Moriarty, B. Kaliski, J. Jonsson, and A. Rusch, “PKCS #1: RSA Cryptography Specifications Version 2.2,” Internet Engineering Task Force, Request for Comments 8017, Nov. 2016, obsoletes RFC 3447. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8017>